Part I:

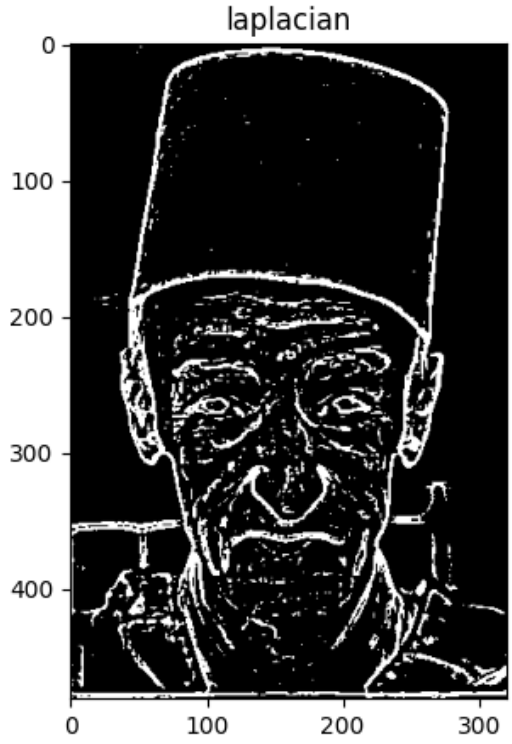
Edge Detection!

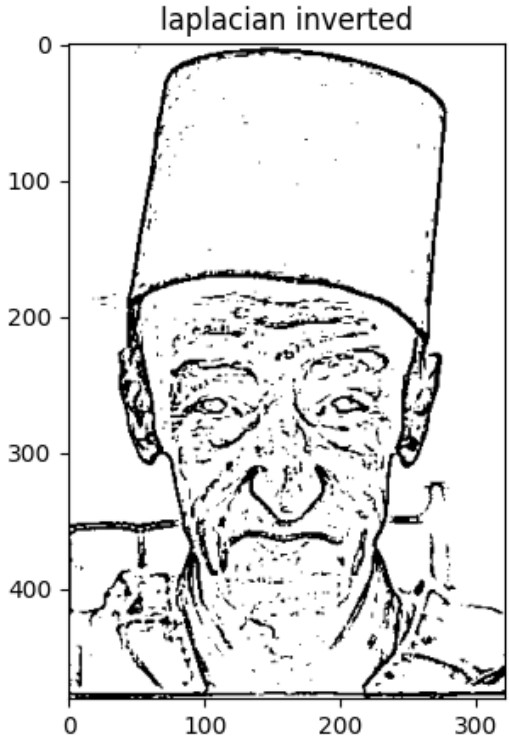
First Filtering!

In order to achieve robust edge detection we first have to process and prepare our images, this is done using a couple of filters namely, something like the median filter that preserves the edges of the image whilst getting rid of the high frequency noise allowing the edge detection algorithm to output a cleaner result.

Second? Getting the derivatives for the Laplacian.

We chose to use the Laplacian filter for edge detection which works by taking the second derivative of the pixel intensities which will then clarify at which pixels the picture heavily shifts frequencies (i.e. an edge pixel)





Third, overlaying our Laplacian-detected edges using a “cartoon-ified” version of the original image.

We create the image to-be overlayed using a bilateral filter which enables us to maintain the edges while maintaining and smoothing out the rest of the pixels, giving us a more “flat/cartoonish” look.

However, the bilateral filter is very expensive, so instead of applying it to the whole image, we chose to down-sample the image and then apply the bilateral filter iteratively instead of having a large kernel-size of 21x21 for example, preserving valuable computational resources.

Finally

We overlayed our previously generated Laplacian-edges, over the newly-generated bilaterally-filtered image to get the final result!

final

Part II

What is the problem we're trying to solve?

Our Edge detection algorithm worked! and it successfully detected the pixels that define those edges.

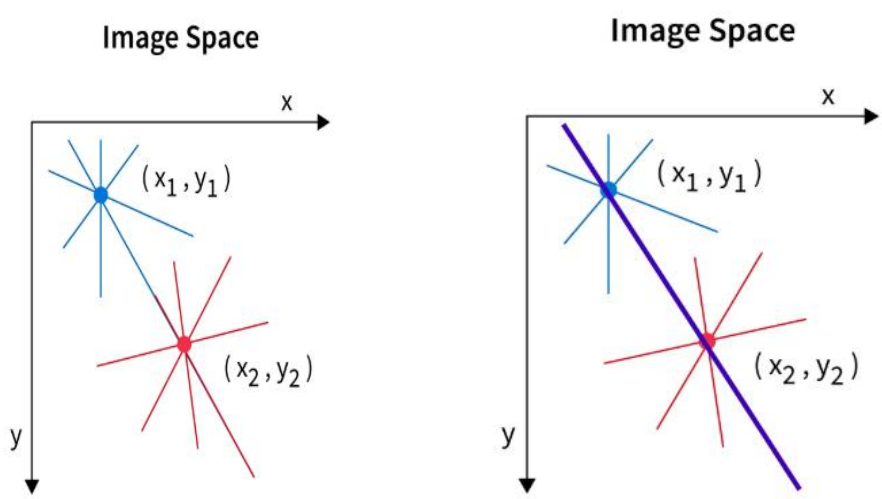
However, there is ****no mathematical correlation between all those collective pixels**** to tell my system that they are all a part of the same line.

Each point in the image space has infinite lines going throug it, however, infinity does NOT mean everything, it just means too many to count.

So, only one of those infinite lines of the first and second pixels are the exact same.

Or, if I have multiple points, only one minimzes the perpendicular distance to each pixel.

But, how do I find this line?



This is where the Hough Transform comes in.

Here is your explanation in plain, standard text formatting:

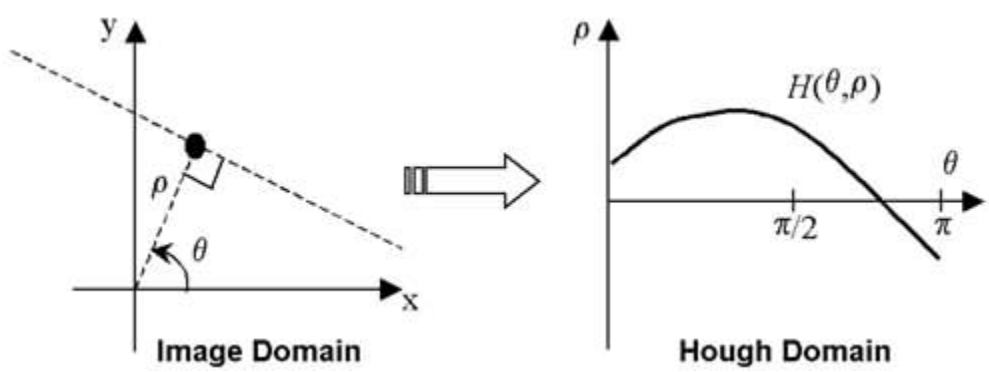
What is the problem we are trying to solve?

Our edge detection algorithm worked! It successfully detected the individual pixels that define those edges. However, there is **no mathematical correlation between all those collective pixels** to tell my system that they are all a part of the same line.

This is where the Hough Transform comes in.

It asks: "Okay, every straight line in my image has a **normal vector** (a perpendicular line connecting it to the origin). That normal vector has a length (**rho**) and an angle (**theta**). I can express the distance from the origin to that entire line using the formula: **rho = x cos(theta) + y sin(theta)**."

Since **Pixel 1** at (x1, y1) could belong to infinite possible lines, I can substitute **theta** from 0 to 180 to map out all the possible (rho, theta) combinations for it. Then I ask: "Which of these specific (rho, theta) pairs will land in the exact same **'bucket'** (the grid cell representing a specific pair of rho and theta) as the pairs I get when I map out all possible lines for **Pixel 2** at (x2, y2)?"

After drawing all the possible values in the **parameter space** for every point given to me by my edge detection algorithm, I see which **rho** and **theta** were used the most or which "grid cell" in the parameter space had the most curves pass through it.

Finally, I use these winning parameters to draw a mathematically perfect line which replaces my old, discrete line of pixels with a newly made **continuous line**.